

Algoritmos y Estructuras de Datos

1er semestre - 2026

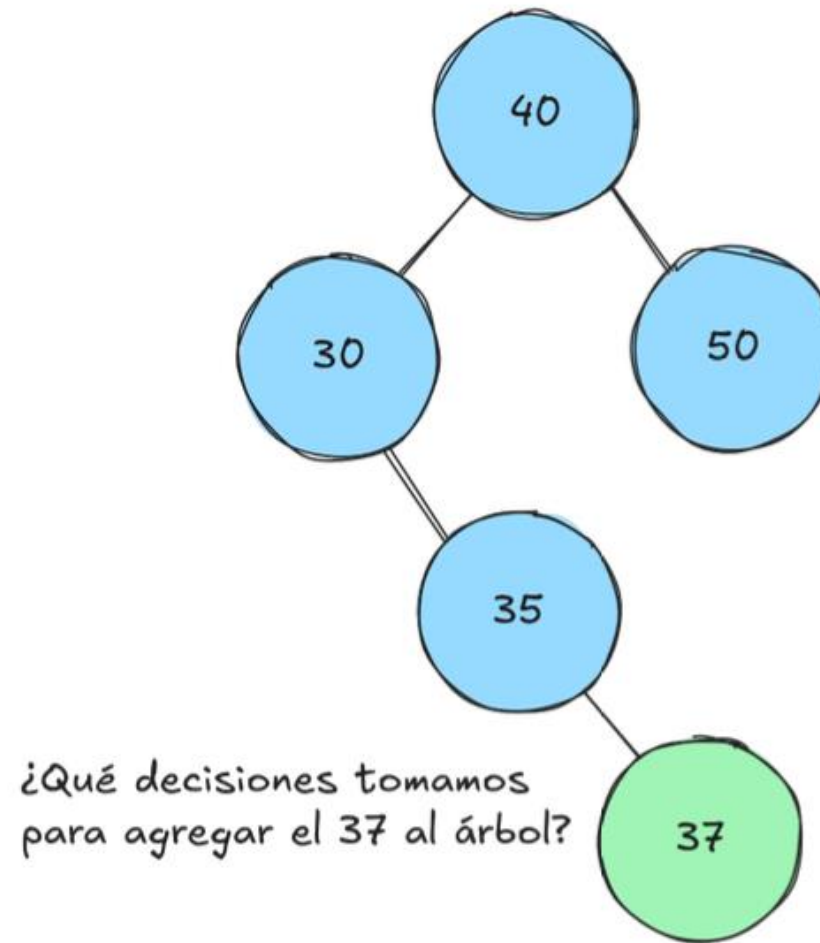
Implementación AVL – Definiciones necesarias

Antes de comenzar, necesitamos una clase AVL que extienda la implementación del ABB.

1. Agregar como método privado **insertarAVL**.
2. Agregar método privado **eliminarAVL** (trabajo domiciliario).
3. NO necesitamos crear un nuevo TElementoAB.

El objetivo de esta clase es extender el comportamiento del ABB y al mismo tiempo reutilizar métodos ya implementados.

Inserción - Procedimiento



Procedimiento

1. Ubicar donde insertar el nuevo dato

- Mismo procedimiento que en ABB

2. Calcular balance

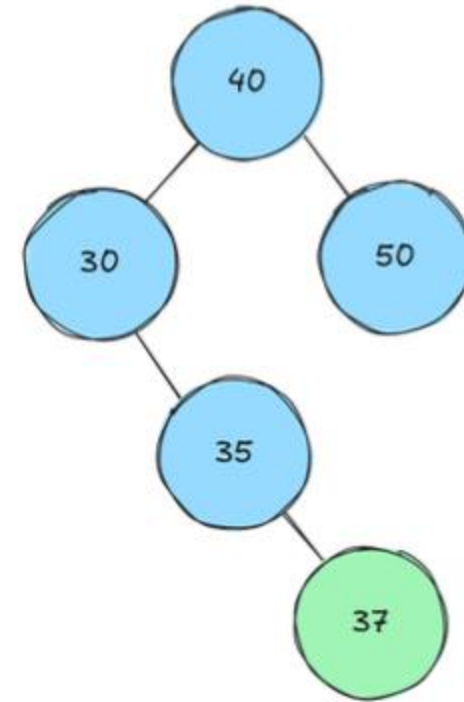
- $\text{balance} = \text{alt}(\text{izq}) - \text{alt}(\text{der})$

3. Con el balance dado, ¿se necesita rotación?

- Aplicar criterio de AVL
- Analizar donde se agregó el dato para determinar tipo de rotación

Procedimiento

- La inserción puede reestructurar un sub-árbol debido a las rotaciones.
- Por esto, necesitamos definir una función insertarAVL que recibe un árbol y retorna otro.
- Esta función puede generar un nuevo árbol ó mantener el anterior.



El sub-árbol izquierdo de 40 no va a ser el mismo luego de corregir el desbalance en 30

Implementación en JAVA

```
public class AVLImpl extends ABBImpl {  
    @Override  
    public boolean insertar(Comparable dato) {  
        raiz = insertarAVL(raiz, dato);  
        return true;  
    }  
  
    private TDAElemento insertarAVL(TDAElemento nodo, Comparable dato) {  
        //TODO Logica de inserción AVL  
        return null;  
    }  
}
```

Pseudocódigo

```
TElementoAB<T> insertarAVL(TElementoAB<T> nodo, T dato){  
    // Caso base  
    1. ???  
    2. ¿Hay más?  
    // Caso recursivo  
    1. Posicionar "dato" en el árbol. Igual que ABB.  
    2. Calcular balance del nodo  
    3. Si hay desbalance, ver donde se agregó el nodo  
       para saber si usar LL, RR, LR, ó RL .  
       Si no hay desbalance devolver nodo.  
}
```

Pseudocódigo

```
TElementoAB<T> insertarAVL(TElementoAB<T> nodo, T dato){  
    // Caso base  
    Si nodo == null  
        retornar nuevo nodo con dato  
    // Caso recursivo  
    1. Posicionar "dato" en el árbol. Igual que ABB.  
    2. Calcular balance del nodo  
    3. Si hay desbalance, ver donde se agregó el nodo  
        para saber si usar LL, RR, LR, ó RL .  
        Si no hay desbalance devolver nodo.  
}
```



```

TElementoAB<T> insertarAVL(TElementoAB<T> nodo, T dato){
    // Caso base
    Si nodo == null
        retornar nuevo nodo con dato
    // Caso recursivo
    // Posicionar "dato" en el árbol. Igual que ABB.
    Si dato < nodo.dato
        nodo.hijoIzq = insertarAVL(nodo.hijoIzq, dato)
    Si dato > nodo.dato
        nodo.hijoDer = insertarAVL(nodo.hijoDer, dato)

    2. Calcular balance del nodo
    3. Si hay desbalance, ver donde se agregó el nodo
        para saber si usar LL, RR, LR, ó RL .
        Si no hay desbalance devolver nodo.
}

```

```

TElementoAB<T> insertarAVL(TElementoAB<T> nodo, T dato){
    // Caso base
    Si nodo == null
        retornar nuevo nodo con dato
    // Caso recursivo
    // Posicionar "dato" en el árbol. Igual que ABB.
    Si dato < nodo.dato
        nodo.hijoIzq = insertarAVL(nodo.hijoIzq, dato)
    Si dato > nodo.dato
        nodo.hijoDer = insertarAVL(nodo.hijoDer, dato)
    // Calcular balance del nodo
    balance = alt(nodo.hijoIzq) - alt(nodo.hijoDer)

    3. Si hay desbalance, ver donde se agregó el nodo
       para saber si usar LL, RR, LR, ó RL .
       Si no hay desbalance devolver nodo.
}

```

```

TElementoAB<T> insertarAVL(TElementoAB<T> nodo, T dato){
    // Caso base
    Si nodo == null
        retornar nuevo nodo con dato
    // Caso recursivo
    // Posicionar "dato" en el árbol. Igual que ABB.
    Si dato < nodo.dato
        nodo.hijoIzq = insertarAVL(nodo.hijoIzq, dato)
    Si dato > nodo.dato
        nodo.hijoDer = insertarAVL(nodo.hijoDer, dato)
    // Calcular balance del nodo
    balance = alt(nodo.hijoIzq) - alt(nodo.hijoDer)
    // Detectar si hay desbalance y tipo de rotación
    Si balance > 1 y dato < nodo.hijoIzq.dato
        retornar rotación LL sobre nodo
    Si balance > 1 y dato > nodo.hijoIzq.dato
        retornar rotación LR sobre nodo
    // pendientes el resto de las rotaciones
    retornar nodo // nodo está balanceado
}

```

Eliminación - Pseudocódigo

```

TElementoAB<T> eliminarAVL(TElementoAB<T> nodo, Comparable<T> criterio){
// Caso base
    Si nodo == null
        retornar null
// Caso recursivo
    Sino
        tmp = nodo // nodo de referencia
        Si criterio.compareTo(nodo.dato) < 0
            nodo.hijoIzq = eliminarAVL(nodo.hijoIzq, criterio)
        Si criterio.compareTo(nodo.dato) > 0
            nodo.hijoDer = eliminarAVL(nodo.hijoDer, criterio)
        Sino
            // en tmp queda el mayor del hijo izquierdo
            tmp = nodo.eliminar(criterio)

        balance = calcBalance(tmp) // definir esta función
        Si balance > 1 y calcBalance(tmp.hijoIzq) >= 0
            retornar rotacion LL sobre tmp
        Si balance > 1 y calcBalance(tmp.hijoIzq) < 0
            retornar rotacion LR sobre tmp
        Si balance < -1 y calcBalance(tmp.hijoDer) <= 0
            retornar rotacion RR sobre tmp
        Si balance < -1 y calcBalance(tmp.hijoDer) > 0
            retornar rotacion RL sobre tmp

        En otro caso, retornar tmp
    }
}

```